CLUBE DE ESTUDOS
BY ELEMAR JR

C#
DO JEITO CERTO

AULA 8
Padrões para Domínios Ricos

CLUBE DE ESTUDOS
BY ELEMAR JR

DESIGN PATTERNS

Tipos de padrões

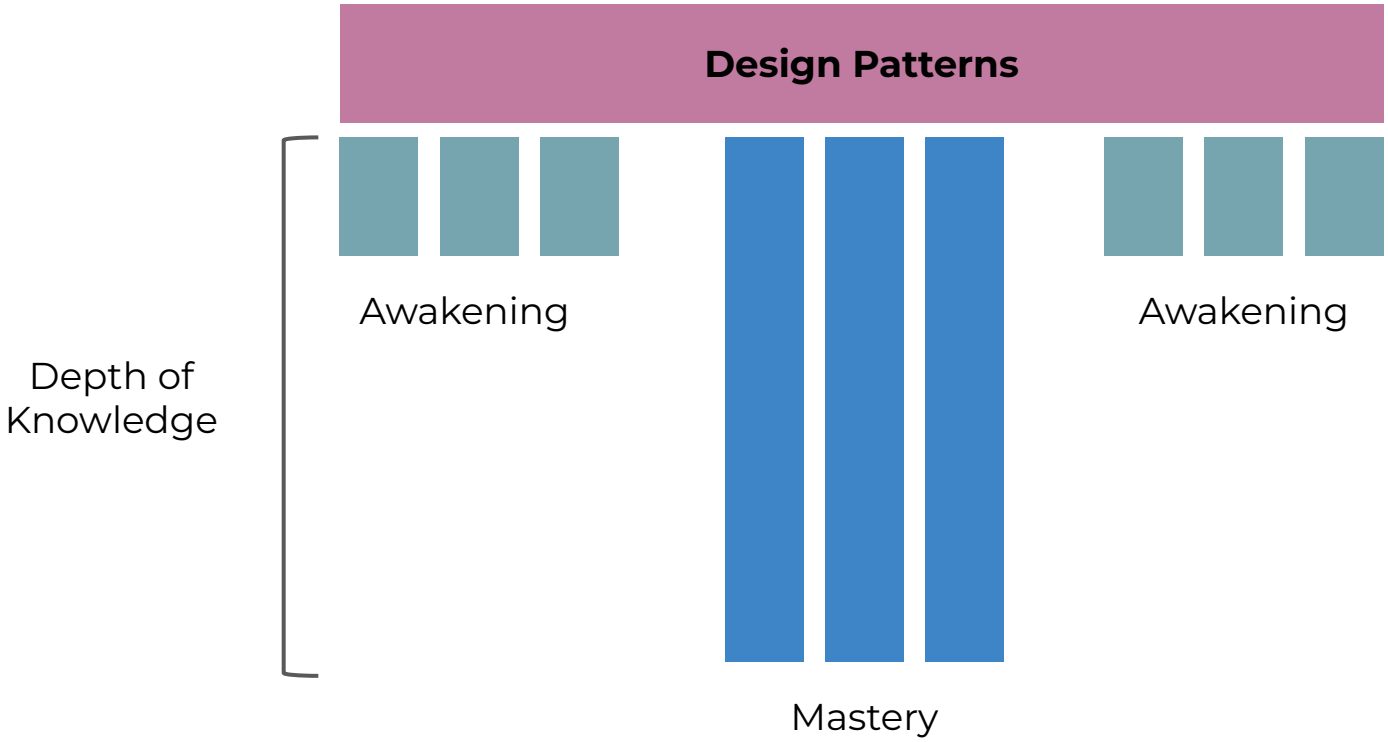
Criacionais: Focam na criação flexível de objetos

Estruturais: Organizam a composição de classes e objetos

Comportamentais: Gerenciam a comunicação entre objetos

Soluções típicas para problemas comuns em design de software. Eles são como receitas que podem ser adaptadas para resolver desafios específicos na programação.

T-Shaped Pattern Knowledge



No dia-a-dia

Praticar

Faça exercícios

Escreva testes e verifique o entendimento

Repita várias vezes com variações

Pratique em código real, em um branch separada (depois deletar)

Código em produção

Siga os princípios de refactoring

Tenha certeza que possui cobertura de testes adequada

Trabalhe em uma branch separada

Verifique se o comportamento é consistente

Esteja preparado para deletar e começar novamente se o resultado não é melhor que o código original

Padrões de projeto devem ser vistos como modelos iniciais e suas implementações podem variar, mas a intenção fundamental de cada padrão deve sempre ser preservada.

CAPÍTULO 1

Creational Design Patterns

O que são ?

Conjunto de padrões que oferecem formas de criar objetos sem utilizar a palavra-chave new explicitamente, garantindo maior flexibilidade e desacoplamento.

FACTORY

Tipos

*Simple Factory**

Factory Method

Abstract Factory

Intenção

Client

Pede um produto

Creator

Facilita a criação

Produto

O produto da criação

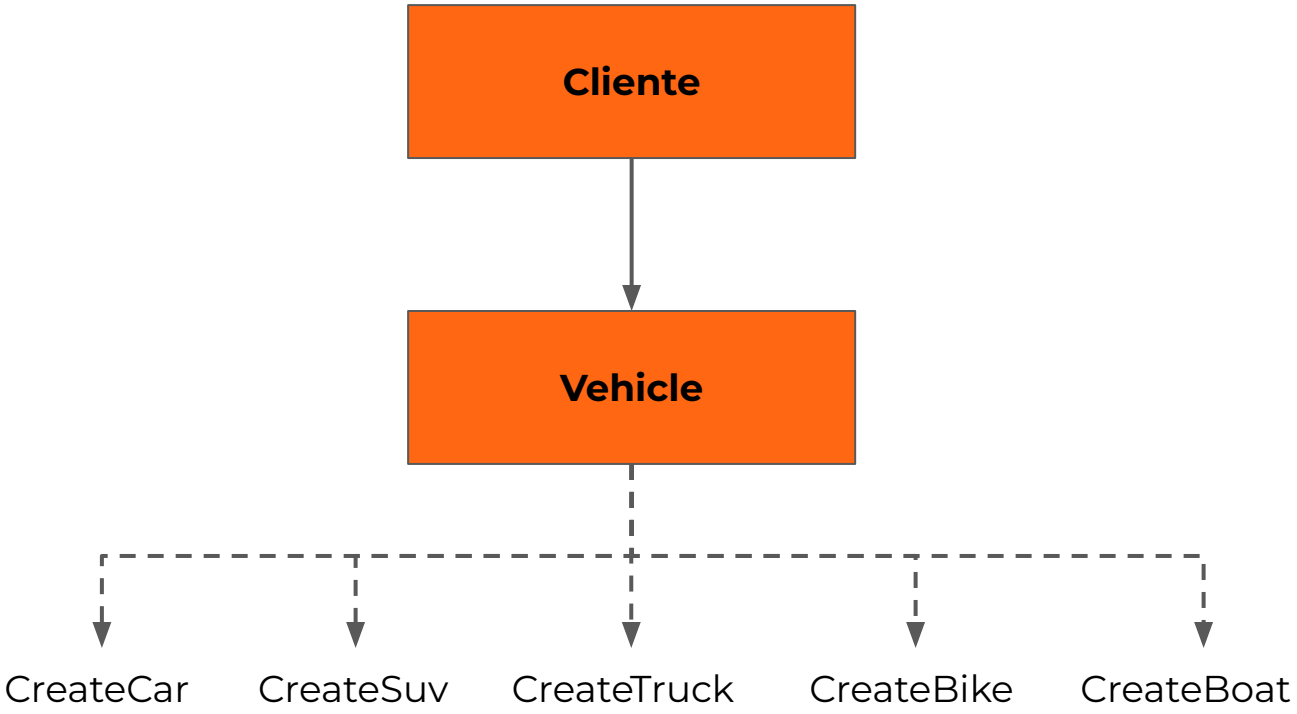
SIMPLE FACTORY

Que dores vamos enfrentar ao realizar manutenção neste código?

```
public Result<string> Finalize()
{
    ShippingProvider shippingProvider;
    if (_order.Sender.Country == "Brazil")
    {
        shippingProvider = new BrazilPostShippingProvider(new ShippingCostCalculator(
            internationalShippingFee: 50,
            extraWeightFee: 100));
    }
    else if (_order.Sender.Country == "EUA")
    {
        shippingProvider = new EUAPostShippingProvider(new ShippingCostCalculator(
            internationalShippingFee: 250,
            extraWeightFee: 500));
    }
    else
        return Result.Failure<string>("Nenhuma transportadora encontrada para o país de origem.");


    _order.PrepareForShipping();
    return shippingProvider.GenerateShippingLabelFor(_order);
}
```

Uma factory é um objeto para criação de outros objetos. Dessa forma, o cliente não precisa mais saber qual objeto criar e de que forma, promovendo maior flexibilidade e extensibilidade ao código.

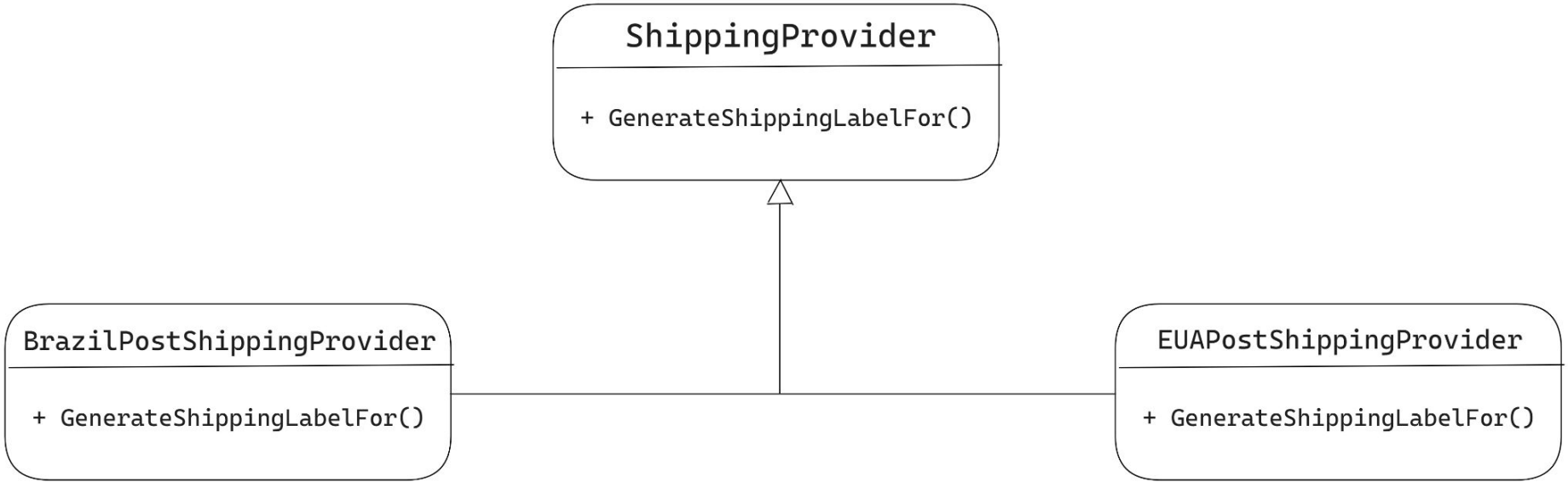
Intenção


Cliente
Vehicle
CreateCar
CreateSuv
CreateTruck
CreateBike
CreateBoat

Processo de finalização de um carrinho de compras

```
public Result<string> Finalize()
{
    ShippingProvider shippingProvider;
    if (_order.Sender.Country == "Brazil")
    {
        shippingProvider = new BrazilPostShippingProvider(new ShippingCostCalculator(
            internationalShippingFee: 50,
            extraWeightFee: 100));
    }
    else if (_order.Sender.Country == "EUA")
    {
        shippingProvider = new EUAPostShippingProvider(new ShippingCostCalculator(
            internationalShippingFee: 250,
            extraWeightFee: 500));
    }
    else
        return Result.Failure<string>("Nenhuma transportadora encontrada para o país de origem.");


    _order.PrepareForShipping();
    return shippingProvider.GenerateShippingLabelFor(_order);
}
```

Processo de finalização de um carrinho de compras



Processo de finalização de um carrinho de compras

```
public Result<string> Finalize()
{
    var shippingProvider = ShippingProvider.Create(_order.Sender.Country);
    if (shippingProvider.IsFailure)
        return Result.Failure<string>(shippingProvider.Error);

    _order.PrepareForShipping();
    return shippingProvider.Value.GenerateShippingLabelFor(_order);
}
```

Processo de finalização de um carrinho de compras

Criação ocorre através de um *static method*

```
public Result<string> Finalize()
{
    var shippingProvider = ShippingProvider.Create(_order.Sender.Country);
    if (shippingProvider.IsFailure)
        return Result.Failure<string>(shippingProvider.Error);

    _order.PrepareForShipping();
    return shippingProvider.Value.GenerateShippingLabelFor(_order);
}
```

Processo de finalização de um carrinho de compras

```
public abstract class ShippingProvider
{
    protected ShippingProvider(ShippingCostCalculator shippingCostCalculator)
    {
        ShippingCostCalculator = shippingCostCalculator;
    }

    public ShippingCostCalculator ShippingCostCalculator { get; protected set; }

    public abstract Result<string> GenerateShippingLabelFor(Order order);

    public static Result<ShippingProvider> Create(string country)
    {
        if (country == "Brazil")
            return new BrazilPostShippingProvider(
                new ShippingCostCalculator(internationalShippingFee: 50, extraWeightFee: 100));

        if (country == "EUA")
            return new EUAPostShippingProvider(
                new ShippingCostCalculator(internationalShippingFee: 250, extraWeightFee: 500));

        return Result.Failure<ShippingProvider>("Nenhuma transportadora encontrada para o país de origem.");
    }
}
```

Processo de finalização de um carrinho de compras

```
public abstract class ShippingProvider
{
    protected ShippingProvider(ShippingCostCalculator shippingCostCalculator)
    {
        ShippingCostCalculator = shippingCostCalculator;
    }

    public ShippingCostCalculator ShippingCostCalculator { get; protected set; }

    public abstract Result<string> GenerateShippingLabelFor(Order order);

    public static Result<ShippingProvider> Create(string country)
    {
        if (country == "Brazil")
            return new BrazilPostShippingProvider(
                new ShippingCostCalculator(internationalShippingFee: 50, extraWeightFee: 100));

        if (country == "EUA")
            return new EUAPostShippingProvider(
                new ShippingCostCalculator(internationalShippingFee: 250, extraWeightFee: 500));

        return Result.Failure<ShippingProvider>("Nenhuma transportadora encontrada para o país de origem.");
    }
}
```

Não é um design pattern formal, mas uma simplificação do Factory Pattern para criação de objetos simples, funcionando muito bem com o envelope Result para validar e controlar a instanciação de forma segura.

Simple Factory

Como saber qual construtor utilizar?

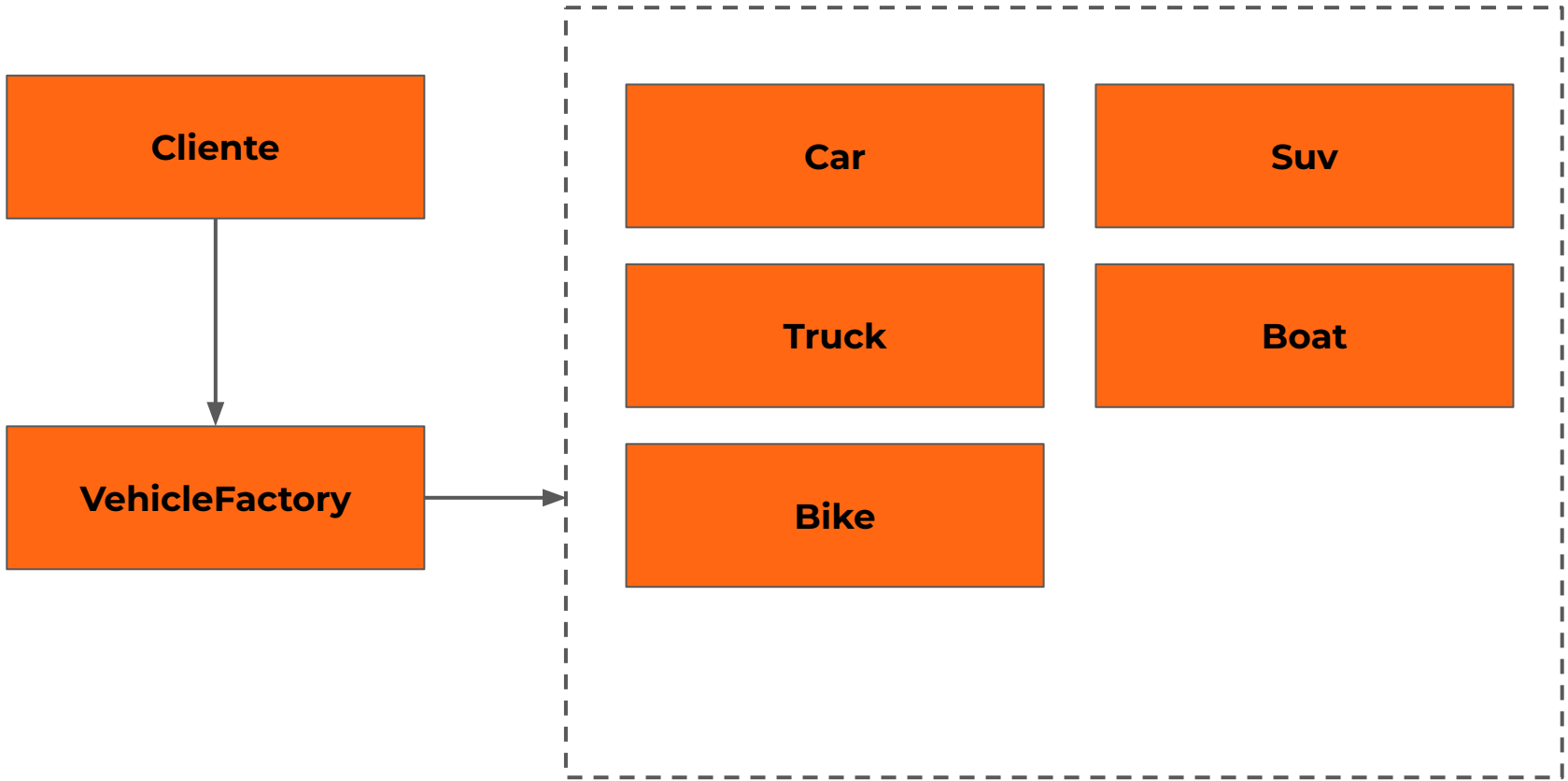
```
public Vehicle (int passengers)
public Vehicle (int passengers, int horsePower)
public Vehicle (int wheels, bool trailer)
public Vehicle (EVehicleType type)
```

Simple Factory

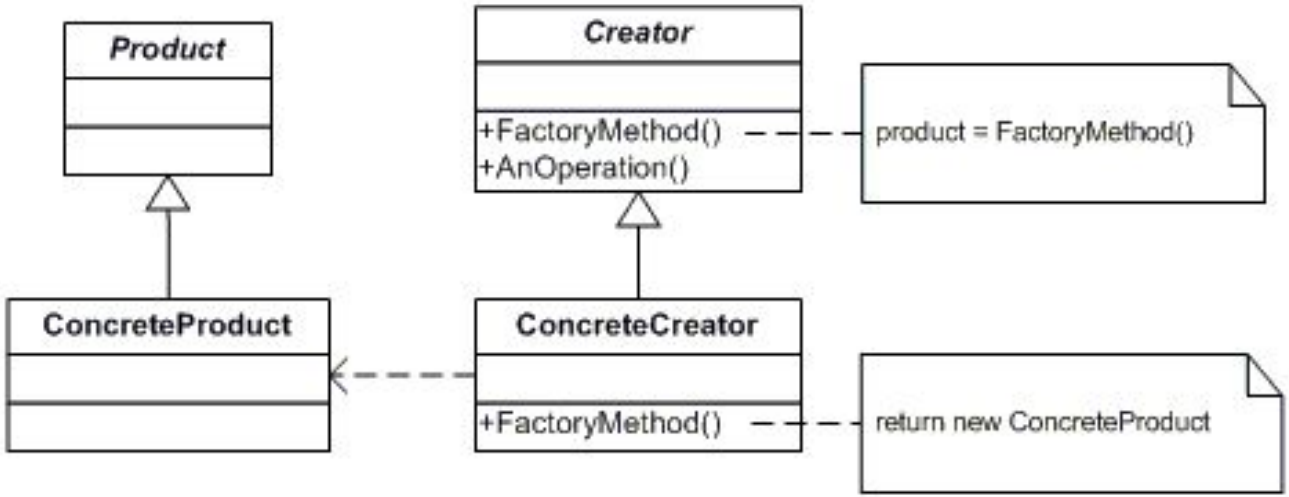

```
public static Vehicle CreateCar(int passengers)
    => new Vehicle(passengers, horsePower: 0, wheels: 0, trailer: false, EVehicleType.Car);

public static Vehicle CreateSuv(int passengers, int horsePower)
    => new Vehicle(passengers, horsePower, wheels: 0, trailer: false, EVehicleType.Suv);

public static Vehicle CreateTruck(int wheels, bool trailer)
    => new Vehicle(passengers: 0, horsePower: 0, wheels, trailer, EVehicleType.Truck);

public static Vehicle CreateBoat()
    => new Vehicle(passengers: 0, horsePower: 0, wheels: 0, trailer: false, EVehicleType.Boat);

public static Vehicle CreateBike()
    => new Vehicle(passengers: 0, horsePower: 0, wheels: 0, trailer: false, EVehicleType.Bike);
```

A intenção do factory method pattern é definir uma interface para a criação de um objeto, mas permitir que as classes derivadas decidam quem instanciar.

Intenção



Factory Method



Creator

Representa o *Creator (abstract factory)*

```
public abstract class ShippingProviderFactory
{
    public abstract ShippingProvider CreateShippingProvider();

    public static Result<ShippingProviderFactory> Create(string country)
    {
        return country switch
        {
            "Brazil" => new BrazilPostShippingProviderFactory(
                new ShippingCostCalculator(internationalShippingFee: 50, extraWeightFee: 100)),
            "EUA" => new EUAPostShippingProviderFactory(
                new ShippingCostCalculator(internationalShippingFee: 250, extraWeightFee: 500)),
            _ => Result.Failure<ShippingProviderFactory>("País não configurado.")
        };
    }
```

Concrete Creator

Representa o *ConcreteCreator(1)*

```
public class BrazilPostShippingProviderFactory : ShippingProviderFactory
{
    private readonly ShippingCostCalculator _shippingCostCalculator;

    public BrazilPostShippingProviderFactory(ShippingCostCalculator shippingCostCalculator)
    {
        _shippingCostCalculator = shippingCostCalculator;
    }

    public override ShippingProvider CreateShippingProvider()
        => new BrazilPostShippingProvider(_shippingCostCalculator);
}
```

Concrete Creator

Representa o *ConcreteCreator(2)*

```
public class EUAPostShippingProviderFactory : ShippingProviderFactory
{
    private readonly ShippingCostCalculator _shippingCostCalculator;

    public EUAPostShippingProviderFactory(ShippingCostCalculator shippingCostCalculator)
    {
        _shippingCostCalculator = shippingCostCalculator;
    }

    public override ShippingProvider CreateShippingProvider()
        => new EUAPostShippingProvider(_shippingCostCalculator);
}
```

Product

Representa o *Product*

```
public abstract class ShippingProvider
{
    protected ShippingProvider(ShippingCostCalculator shippingCostCalculator)
    {
        ShippingCostCalculator = shippingCostCalculator;
    }

    public ShippingCostCalculator ShippingCostCalculator { get; protected set; }

    public abstract Result<string> GenerateShippingLabelFor(Order order);
}
```

Concrete Creator

Representa o Concrete*Product(1)*

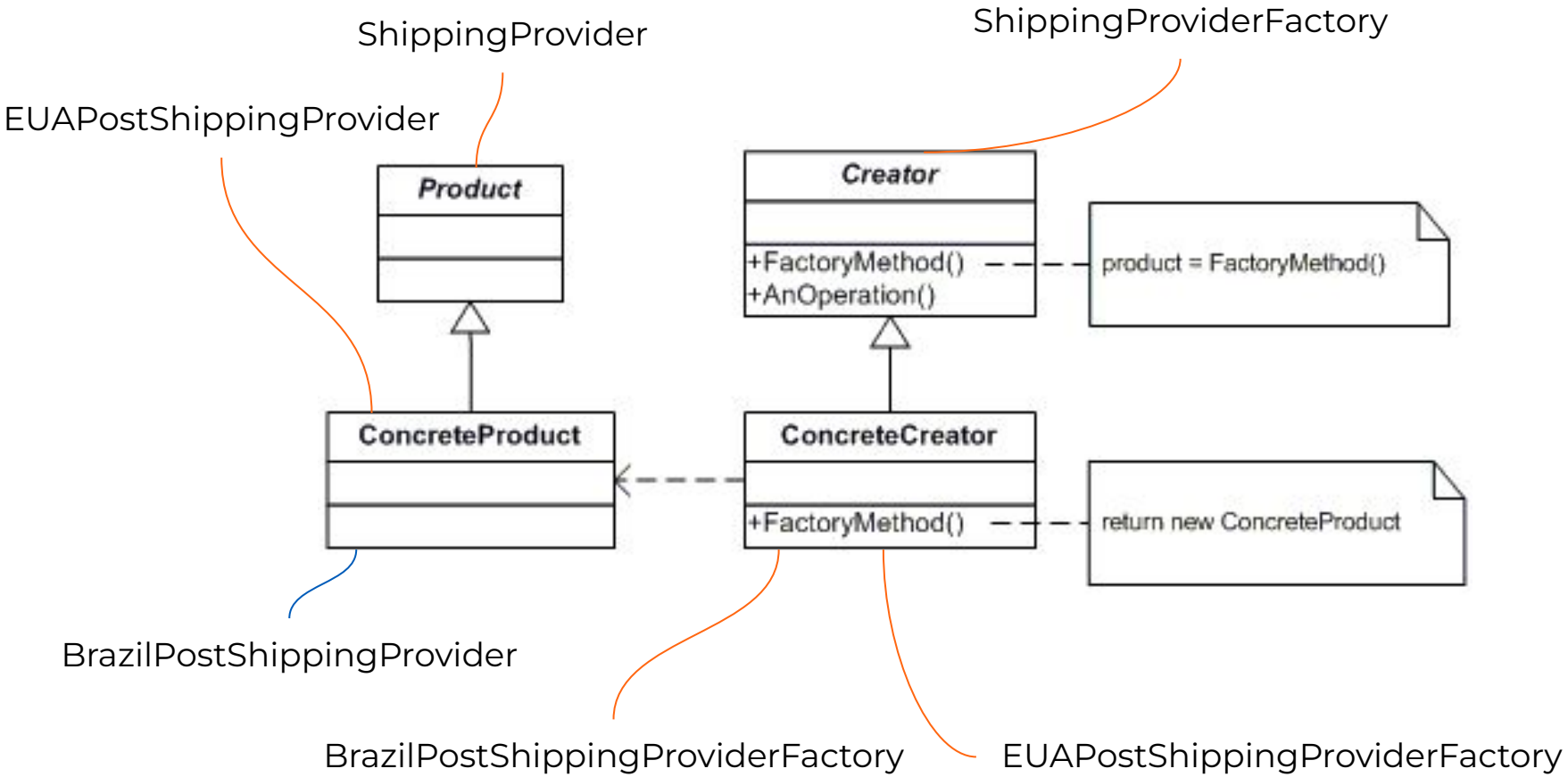
```
public class BrazilPostShippingProvider : ShippingProvider
{
    public BrazilPostShippingProvider(ShippingCostCalculator shippingCostCalculator)
        : base(shippingCostCalculator) { }

    public override Result<string> GenerateShippingLabelFor(Order order)
    {
        if (order.Recipient.Country != order.Sender.Country)
            return Result.Failure<string>("Envio internacional não suportado.");

        var shippingCost = ShippingCostCalculator.CalculateFor(
            order.Recipient.Country,
            order.Sender.Country,
            order.TotalWeight);

        return
            $"Brazil post shipping {Environment.NewLine}" +
            $"To: {order.Recipient.To} {Environment.NewLine}" +
            $"Order total: {order.Total} {Environment.NewLine}" +
            $"Shipping Cost: {shippingCost}";
    }
}
```

Concrete Creator

Representa o Concrete*Product(2)*

```
public class EUAPostShippingProvider : ShippingProvider
{
    public EUAPostShippingProvider(ShippingCostCalculator shippingCostCalculator)
        : base(shippingCostCalculator) { }

    public override Result<string> GenerateShippingLabelFor(Order order)
    {
        var shippingCost = ShippingCostCalculator.CalculateFor(
            order.Recipient.Country,
            order.Sender.Country,
            order.TotalWeight);

        return
            $"EUA post shipping {Environment.NewLine}" +
            $"To: {order.Recipient.To} {Environment.NewLine}" +
            $"Order total: {order.Total} {Environment.NewLine}" +
            $"Shipping Cost: {shippingCost}";
    }
}
```

Factory Method



Factory Method


```
public class ShoppingCart
{
    private readonly Order _order;

    public ShoppingCart(Order order)
    {
        _order = order;
    }

    public Result<string> Finalize()
    {
        var shippingProviderFactory = ShippingProviderFactory.Create(_order.Sender.Country);
        if (shippingProviderFactory.IsFailure)
            return Result.Failure<string>(shippingProviderFactory.Error);

        var shippingProvider = shippingProviderFactory.Value.CreateShippingProvider();

        _order.PrepareForShipping();
        return shippingProvider.GenerateShippingLabelFor(_order);
    }
}
```

Open/Closed Principle

Atende ao open/closed principle, pois novas transportadoras podem ser adicionadas sem quebrar o código cliente

Single Responsibility Principle

Atende ao single responsibility principle, pois a criação de transportadoras está encapsulada em um local

É necessário implementar uma subclasse para cada novo ConcreteProduct

Uso no .NET

- Utilizado na **System.Convert**. Um determinado tipo é recebido e um novo é retornado
- Utilizado no **File.Open**

- https://github.com/microsoft/referencesource/blob/master/mscorlib/system/convert.cs#L385
- https://github.com/microsoft/referencesource/blob/master/mscorlib/system/io/file.cs

Factory Method no System.Convert

Recebe um tipo e retorna outro

```
Convert.ToBoolean("true");
Convert.ToString(123456);
Convert.ToInt32("657");
```

Factory Method na classe File


```
var path = @"c:\temp\MyTest.txt";

if (!File.Exists(path))
{
    using FileStream fs01 = File.Create(path);
    var info = new UTF8Encoding(true).GetBytes("This is some text in the file.");

    fs01.Write(info, 0, info.Length);
}

using FileStream fs02 = File.OpenRead(path);

byte[] b = new byte[1024];
UTF8Encoding temp = new UTF8Encoding(true);

while (fs02.Read(b, 0, b.Length) > 0)
    Console.WriteLine(temp.GetString(b));
```

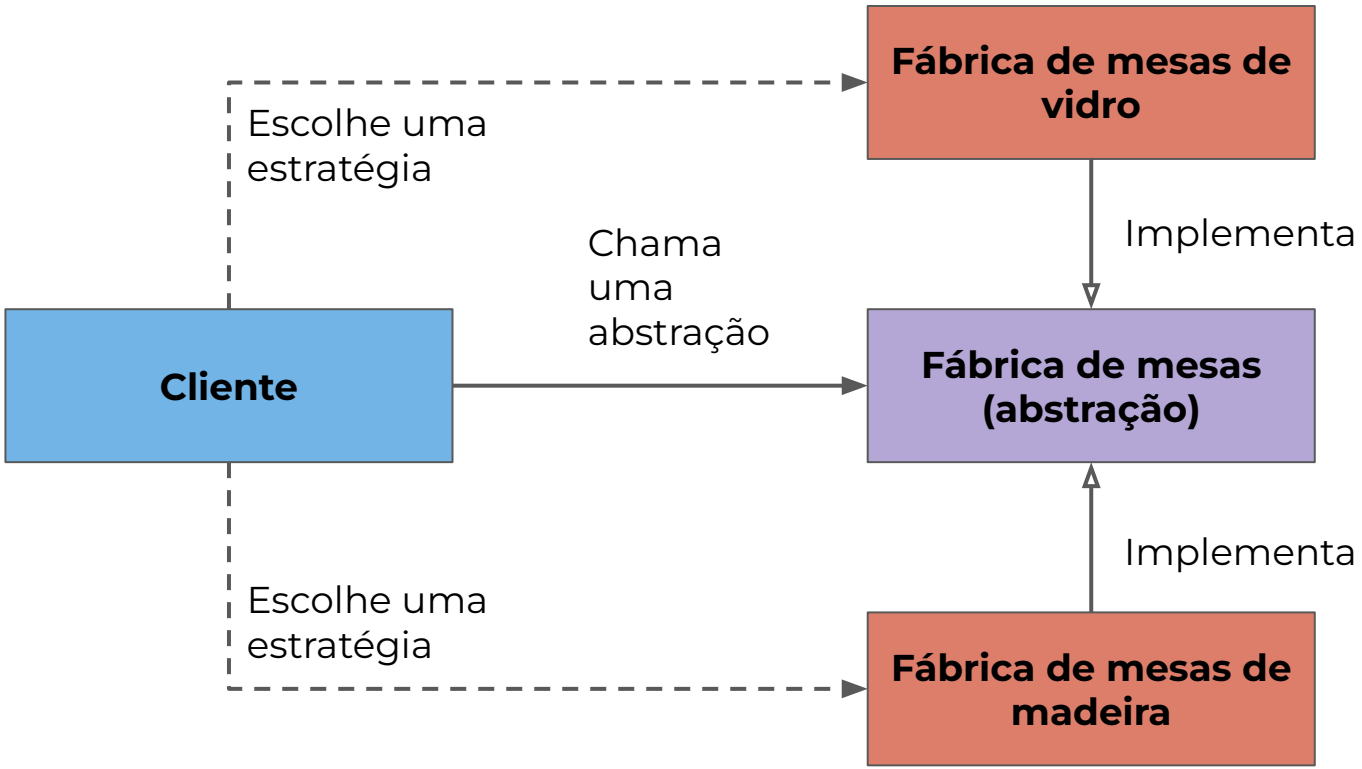
Factory Method recebe o *path* e cria um *FileStream* para escrita

Factory Method recebe o *path* e cria um *FileStream*, o abrindo para leitura

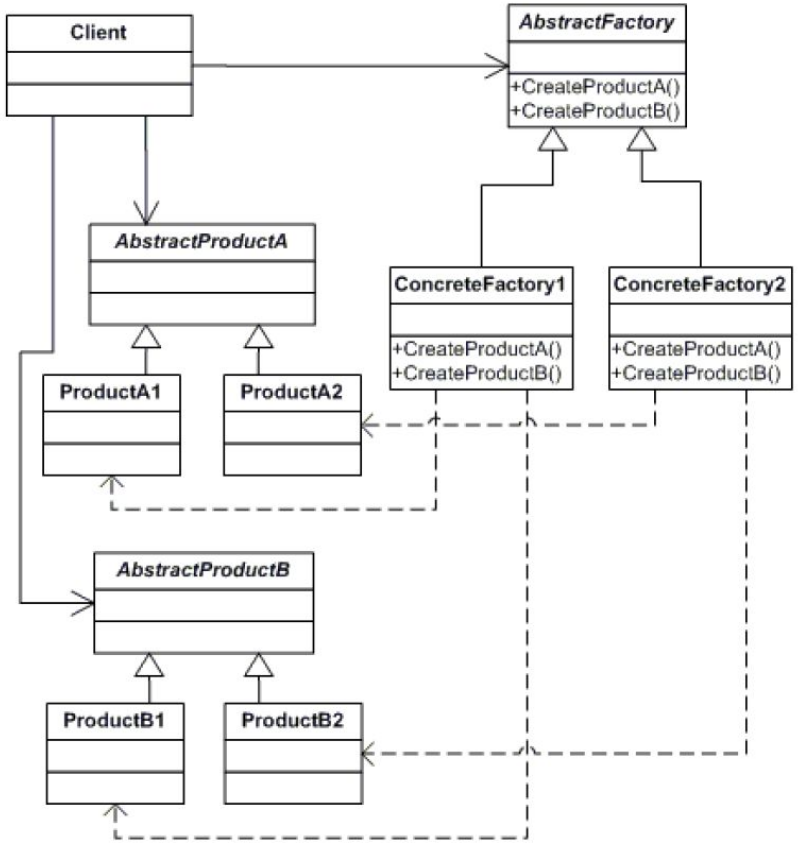
ABSTRACT FACTORY

A intenção do abstract factory pattern é prover uma interface para criação de uma família de objetos de forma abstraída, sem especificar suas implementações.

Intenção


Fábrica de mesas de vidro
Escolhe uma estratégia
Implementa
Chama uma abstração
Cliente
Fábrica de mesas (abstração)
Implementa
Escolhe uma estratégia
Fábrica de mesas de madeira

Abstract Factory



Abstract Factory

Representa o *AbstractFactory*

```
public interface IShoppingCartPurchaseFactory
{
    IShippingLabelService CreateLabelGenerationService();
    IShippingCostService CreateCostCalculationService();
}
```

Trabalhando como *interface*, não mais herança

Abstract Factory

CLUBE DE ESTUDOS
BY ELEMAR JR

```
public interface IShippingCostService
{
    decimal CalculateFor(decimal weight);
}


public interface IShippingLabelService
{
    Result<string> GenerateFor(Order order, decimal shippingCost);
}
```

AbstractProduct's

Concrete Product


```
public class BrazilCostService : IShippingCostService
{
    public decimal CalculateFor(decimal weight)
        => 0;
}


public class BrazilLabelService : IShippingLabelService
{
    public Result<string> GenerateFor(Order order, decimal shippingCost)
    {
        if (order.Recipient.Country != order.Sender.Country)
            return Result.Failure<string>("Envio internacional não suportado.");

        return
            $"Brazil post shipping {Environment.NewLine}" +
            $"To: {order.Recipient.To} {Environment.NewLine}" +
            $"Order total: {order.Total} {Environment.NewLine}" +
            $"Shipping Cost: {shippingCost}";
    }
}
```

ConcreteProduct's (1)

Concrete Product


```
public class InternationalCostService : IShippingCostService
{
    public decimal CalculateFor(decimal weight)
        => 100;
}
```

```
public class InternationalLabelService : IShippingLabelService
{
    public Result<string> GenerateFor(Order order, decimal shippingCost)
        => $"International post shipping {Environment.NewLine}" +
            $"To: {order.Recipient.To} {Environment.NewLine}" +
            $"Order total: {order.Total} {Environment.NewLine}" +
            $"Shipping Cost: {shippingCost}";
}
```

ConcreteProduct's (2)

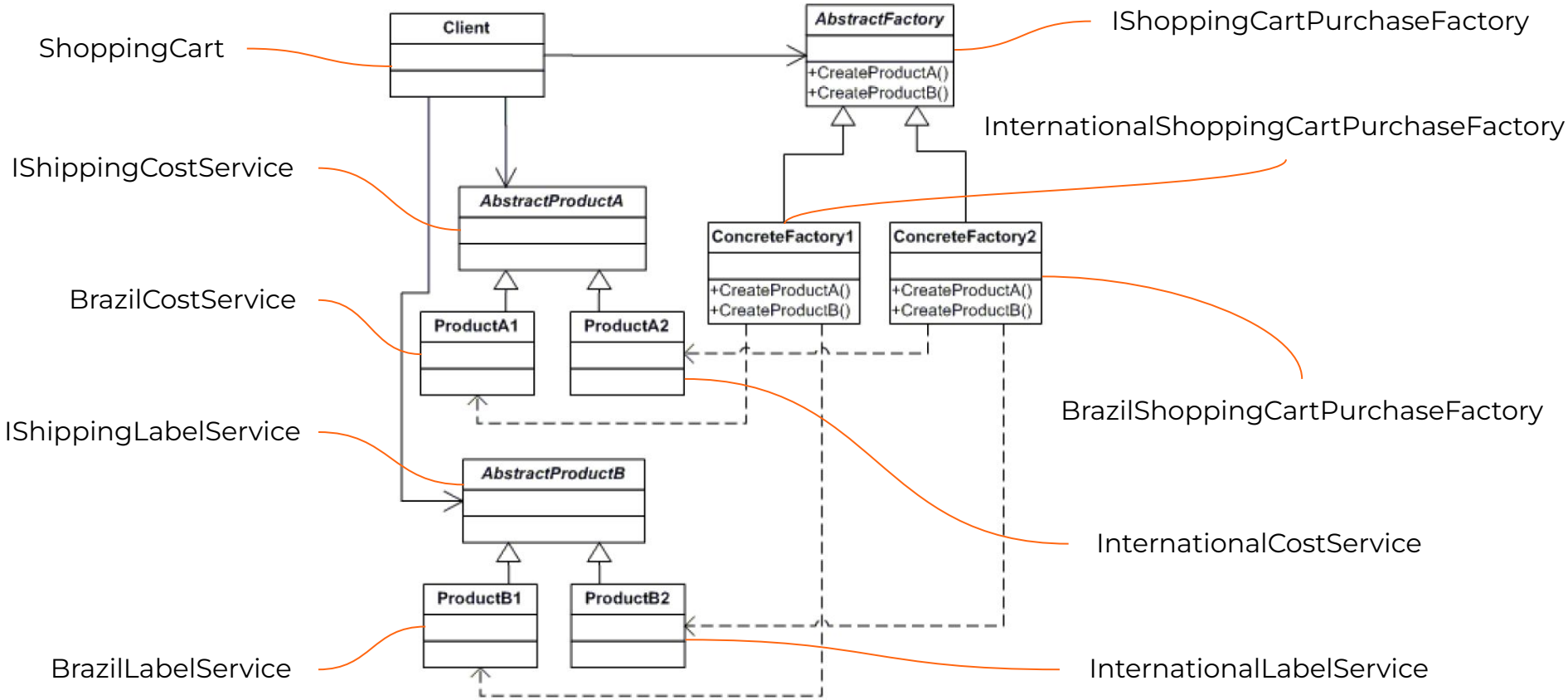
Concrete Factory


```
public class BrazilShoppingCartPurchaseFactory : IShoppingCartPurchaseFactory
{
    public IShippingCostService CreateCostCalculationService()
        => new BrazilCostService();

    public IShippingLabelService CreateLabelGenerationService()
        => new BrazilLabelService();
}


public class InternationalShoppingCartPurchaseFactory : IShoppingCartPurchaseFactory
{
    public IShippingCostService CreateCostCalculationService()
        => new InternationalCostService();

    public IShippingLabelService CreateLabelGenerationService()
        => new InternationalLabelService();
}
```

ConcreteFactories

Abstract Factory

CLUBE DE ESTUDOS
BY ELEMAR JR



Abstract Factory


```
public class ShoppingCart
{
    private readonly Order _order;
    private readonly IShippingCostService _shippingCostService;
    private readonly IShippingLabelService _shippingLabelService;

    public ShoppingCart(Order order, IShoppingCartPurchaseFactory shoppingCartPurchaseFactory)
    {
        _order = order;
        _shippingCostService = shoppingCartPurchaseFactory.CreateCostCalculationService();
        _shippingLabelService = shoppingCartPurchaseFactory.CreateLabelGenerationService();
    }

    public Result<string> Finalize()
    {
        _order.PrepareForShipping();

        var shippingCost = _shippingCostService.CalculateFor(_order.TotalWeight);
        return _shippingLabelService.GenerateFor(_order, shippingCost);
    }
}
```

Abstract Factory

Faz uso de ambos serviços criados

```
public class ShoppingCart
{
    private readonly Order _order;
    private readonly IShippingCostService _shippingCostService;
    private readonly IShippingLabelService _shippingLabelService;

    public ShoppingCart(Order order, IShoppingCartPurchaseFactory shoppingCartPurchaseFactory)
    {
        _order = order;
        _shippingCostService = shoppingCartPurchaseFactory.CreateCostCalculationService();
        _shippingLabelService = shoppingCartPurchaseFactory.CreateLabelGenerationService();
    }

    public Result<string> Finalize()
    {
        _order.PrepareForShipping();

        var shippingCost = _shippingCostService.CalculateFor(_order.TotalWeight);
        return _shippingLabelService.GenerateFor(_order, shippingCost);
    }
}
```

Facilita a troca de família de produtos, uma vez que há uma factory por família

Suportar novos tipos de produtos pode ser bastante difícil, uma vez que implicaria em alterar a abstract factory

Abstract Factory no ADO.NET


```
public abstract class DbProviderFactory
{
    protected DbProviderFactory() { }

    virtual public bool CanCreateDataSourceEnumerator
    {
        get { return false; }
    }

    public virtual DbCommand CreateCommand() => null!;
    public virtual DbCommandBuilder CreateCommandBuilder() => null!;
    public virtual DbConnection CreateConnection() => null!;
    public virtual DbConnectionStringBuilder CreateConnectionStringBuilder() => null!;
    public virtual DbDataAdapter CreateDataAdapter() => null!;
    public virtual DbParameter CreateParameter() => null!;
    public virtual CodeAccessPermission CreatePermission(PermissionState state) => null!;
    public virtual DbDataSourceEnumerator CreateDataSourceEnumerator() => null!;
}
```

BUILDER

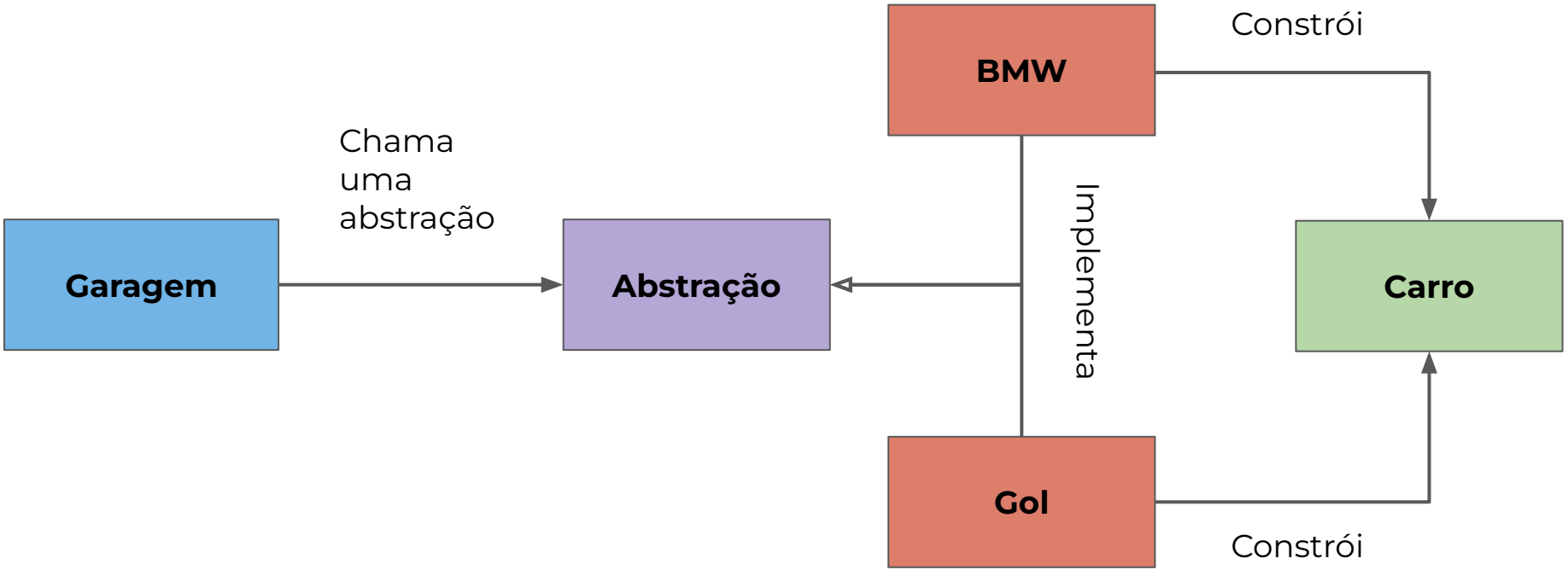
Que dores vamos enfrentar ao realizar manutenção neste código?

CLUBE DE ESTUDOS
BY ELEMAR JR

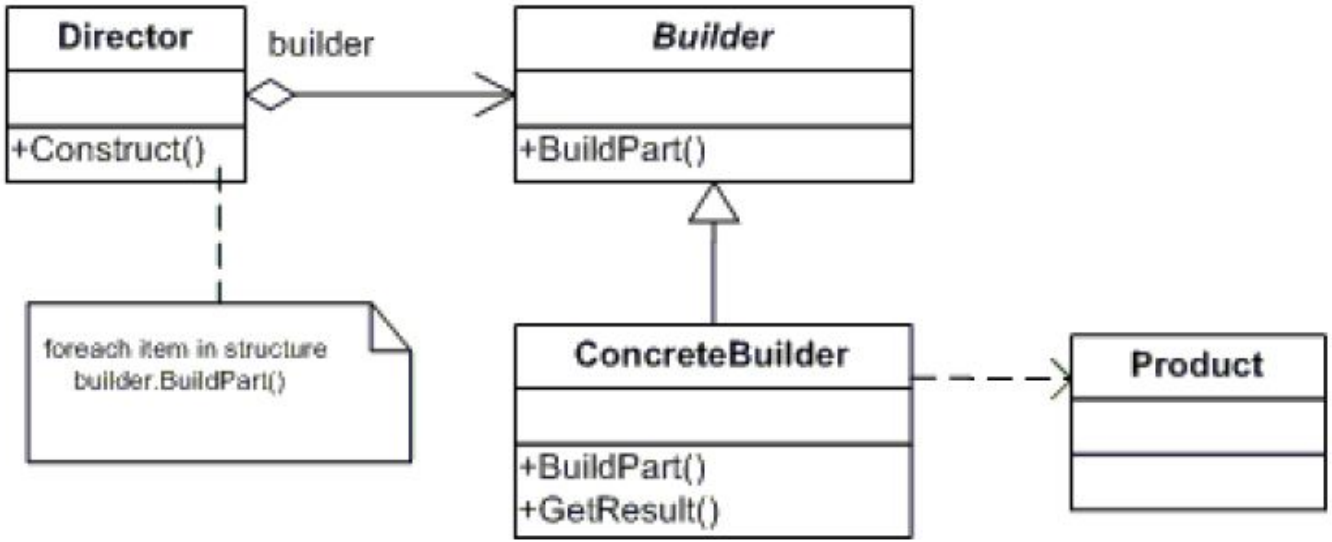
```
public class Vehicle
{
    public Vehicle(EVehicleType type)
    public Vehicle(int passengers, string engine)
    public Vehicle(int wheels, int passengers)
    public Vehicle(int passengers, string engine, bool trailer)
    public Vehicle(EVehicleType type, int wheels, int passengers, string engine, bool trailer)

    public EVehicleType Type { get; }
    public int Wheels { get; }
    public int Passengers { get; }
    public string Engine { get; }
    public bool Trailer { get; }
}
```

A intenção do builder pattern é separar a construção de um objeto complexo das suas representações, ou seja, possibilitar que ele seja construído step by step. O mesmo processo de construção pode criar diferentes representações.

Intenção


BMW
Constrói
Chama
uma
abstração
Garagem
Abstração
Implementa
Carro
Gol
Constrói

Abstract Factory



Product

Representa o *Product*

```
public class Vehicle
{
    private readonly Dictionary<EPartType, string> _parts = new();
    private readonly EVehicleType _vehicleType;

    public Vehicle(EVehicleType vehicleType)
    {
        _vehicleType = vehicleType;
    }

    public string this[EPartType key]
    {
        get { return _parts[key]; }
        set { _parts[key] = value; }
    }

    public void Show()
    {
        Console.WriteLine("\n---------------------------");
        Console.WriteLine($"Vehicle Type: {_vehicleType}");
        Console.WriteLine($" Frame : {this[EPartType.Frame]}");
        Console.WriteLine($" Engine: {this[EPartType.Engine]}");
        Console.WriteLine($" Wheels: {this[EPartType.Wheel]}");
        Console.WriteLine($" Doors : {this[EPartType.Door]}");
    }
}
```

Representa o *Builder*

```
public abstract class VehicleBuilder(EVehicleType vehicleType)
{
    public Vehicle Vehicle { get; private set; } = new Vehicle(vehicleType);

    public abstract void BuildFrame();
    public abstract void BuildEngine();
    public abstract void BuildWheels();
    public abstract void BuildDoors();
}
```

Concrete Builder

Representa o *ConcreteBuilder(1)*

```
public class CarBuilder : VehicleBuilder
{
    public CarBuilder() : base(EVehicleType.Car)
    { }

    public override void BuildFrame() => Vehicle[EPartType.Frame] = "Car Frame";
    public override void BuildEngine() => Vehicle[EPartType.Engine] = "2500 cc";
    public override void BuildWheels() => Vehicle[EPartType.Wheel] = "4";
    public override void BuildDoors() => Vehicle[EPartType.Door] = "4";
}
```

Concrete Builder

Representa o *ConcreteBuilder(2)*

```
public class MotorCycleBuilder : VehicleBuilder
{
    public MotorCycleBuilder() : base(EVehicleType.MotorCycle)
    { }

    public override void BuildFrame() => Vehicle[EPartType.Frame] = "MotorCycle Frame";
    public override void BuildEngine() => Vehicle[EPartType.Engine] = "500 cc";
    public override void BuildWheels() => Vehicle[EPartType.Wheel] = "2";
    public override void BuildDoors() => Vehicle[EPartType.Door] = "0";
}
```

Concrete Builder

Representa o *ConcreteBuilder(3)*

```
public class ScooterBuilder : VehicleBuilder
{
    public ScooterBuilder() : base(EVehicleType.Scooter)
    { }

    public override void BuildFrame() => Vehicle[EPartType.Frame] = "Scooter Frame";
    public override void BuildEngine() => Vehicle[EPartType.Engine] = "50 cc";
    public override void BuildWheels() => Vehicle[EPartType.Wheel] = "2";
    public override void BuildDoors() => Vehicle[EPartType.Door] = "0";
}
```

Director


```
public class Shop
{
    private VehicleBuilder? _builder;

    public void Construct(VehicleBuilder vehicleBuilder)
    {
        _builder = vehicleBuilder;

        _builder.BuildFrame();
        _builder.BuildEngine();
        _builder.BuildWheels();
        _builder.BuildDoors();
    }

    public void ShowVehicle()
    {
        _builder?.Vehicle.Show();
    }
}
```

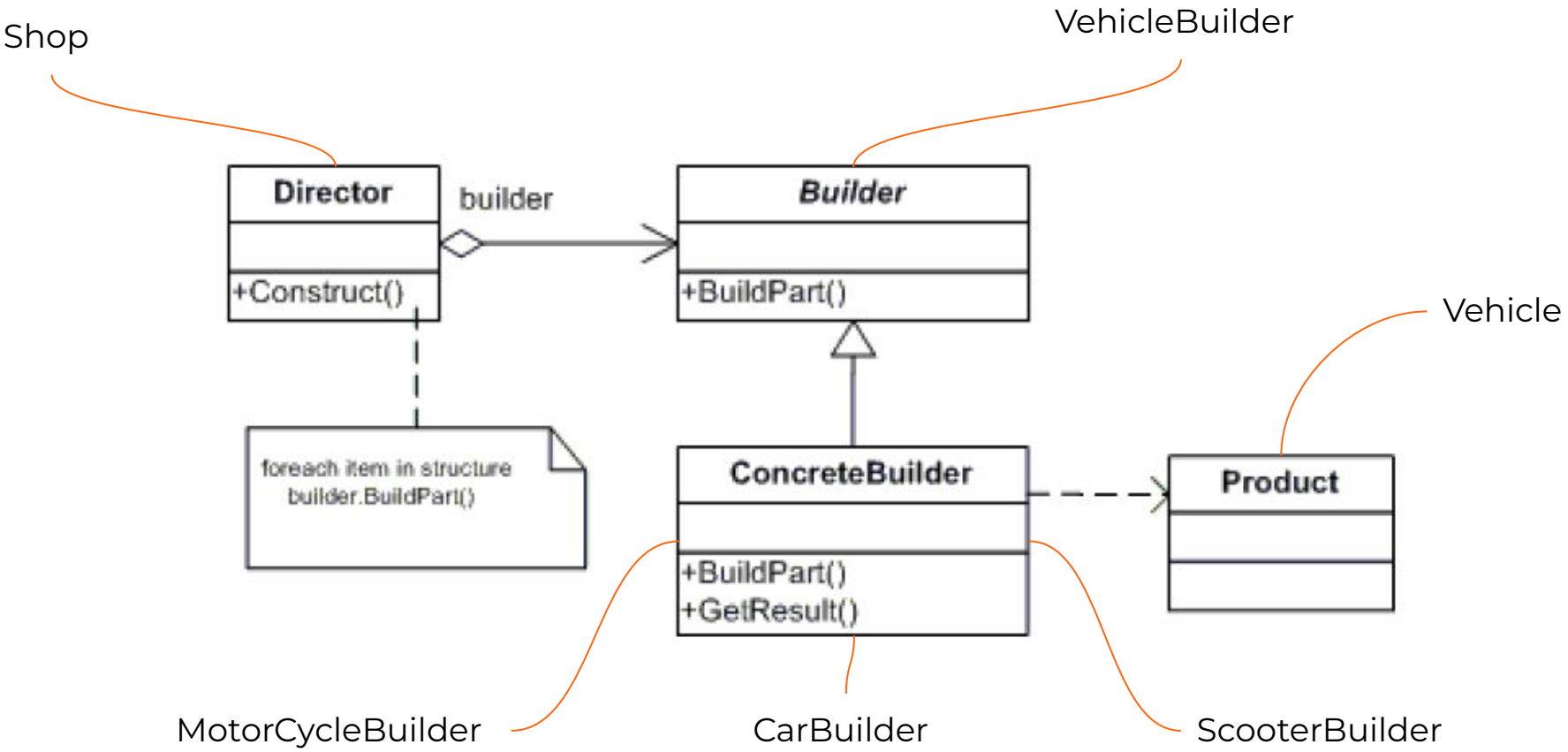
Representa o *Director*

Director


```
public class Shop
{
    private VehicleBuilder? _builder;

    public void Construct(VehicleBuilder vehicleBuilder)
    {
        _builder = vehicleBuilder;

        _builder.BuildFrame();
        _builder.BuildEngine();
        _builder.BuildWheels();
        _builder.BuildDoors();
    }

    public void ShowVehicle()
    {
        _builder?.Vehicle.Show();
    }
}
```

Efetua o *build* do objeto

Builder


Shop
VehicleBuilder
Director
builder
Builder
+Construct()
+BuildPart()
Vehicle
foreach item in structure
builder.BuildPart()
ConcreteBuilder
Product
+BuildPart()
+GetResult()
MotorCycleBuilder
CarBuilder
ScooterBuilder

Builder Pattern

Constrói o veículo correspondente ao *builder* instanciado

```
Shop shop = new();

shop.Construct(new ScooterBuilder());
shop.ShowVehicle();

shop.Construct(new CarBuilder());
shop.ShowVehicle();

shop.Construct(new MotorCycleBuilder());
shop.ShowVehicle();
```

Atende ao single responsibility principle, pois promove a construção e representação de objetos complexos

Promove um controle fino sobre o processo de construção

Pode levar a um aumento da complexidade do código

Builder nas Configurações de API


```
var builder = WebApplication.CreateBuilder(args);

builder
    .Configuration
    .SetBasePath(Directory.GetCurrentDirectory())
    .AddJsonFile("appsettings.json", optional: false, reloadOnChange: true)
    .AddUserSecrets<Program>()
    .AddEnvironmentVariables();
```

Builder nas Queries do EF Core


```
return await _dbContext
    .Signatures
    .Include(s => s.Beneficiaries)
    .Where(s => s.Status == ESignatureStatus.Active
        && s.HolderId == holderId)
    .ToListAsync(cancellationToken)
    .ConfigureAwait(false);
```

Builder no StringBuilder


```
StringBuilder stringBuilder = new();
stringBuilder
    .Append("Teste")
    .AppendLine("Teste 02")
    .AppendLine("Teste 03")
    .ToString();
```

Builder no Serilog


```
Log.Logger = new LoggerConfiguration()
    .MinimumLevel.Information()
    .MinimumLevel.Override("Microsoft", LogEventLevel.Error)
    .MinimumLevel.Override("System", LogEventLevel.Error)
    .MinimumLevel.Override("Pivotal", LogEventLevel.Error)
    .MinimumLevel.Override("Steeltoe", LogEventLevel.Error)
    .MinimumLevel.Override("Azure", LogEventLevel.Error)
    .Enrich.FromLogContext()
    .WriteTo.Console()
    .CreateLogger();
```

SINGLETON

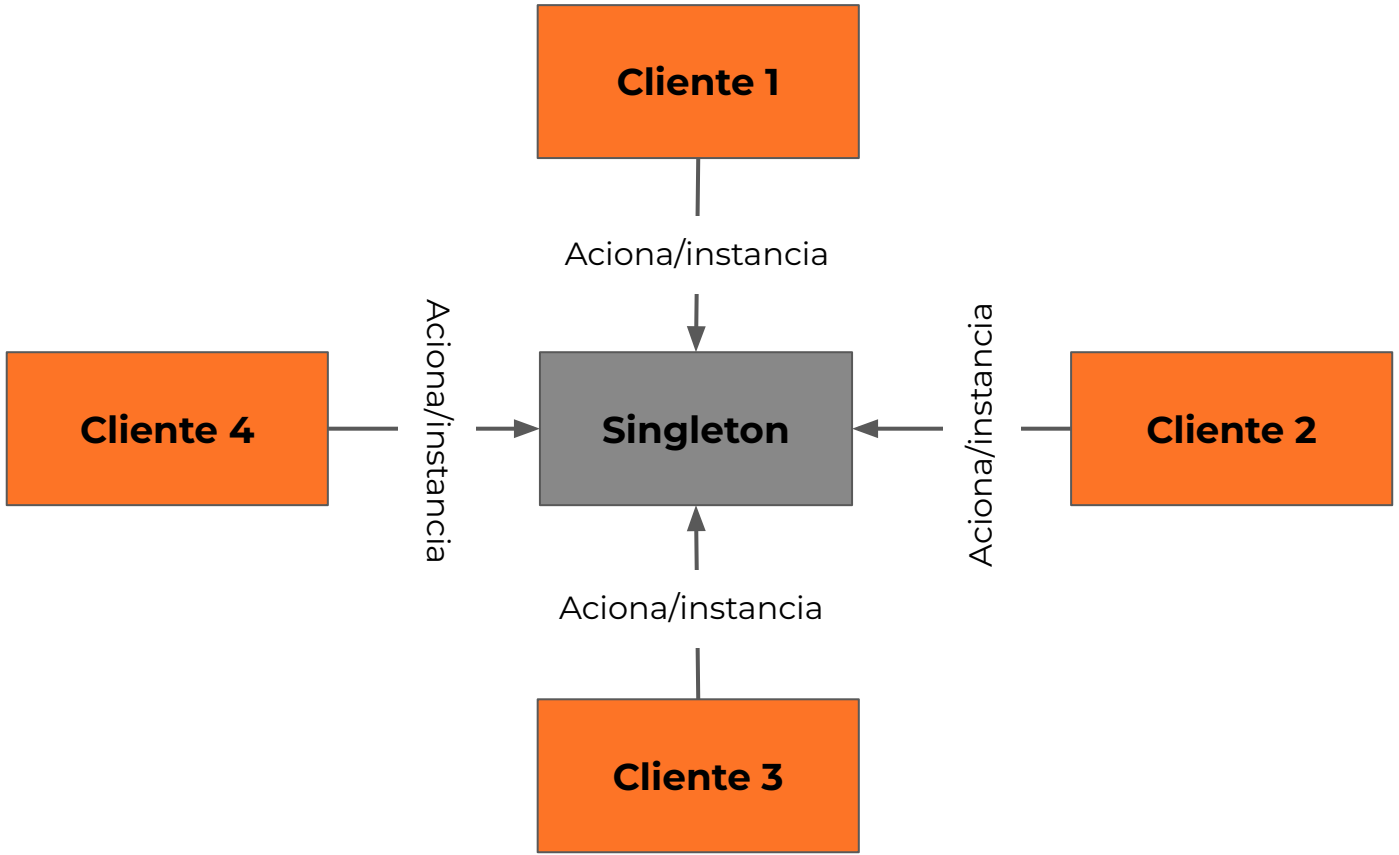
Que dores vamos enfrentar ao realizar manutenção neste código?

CLUBE DE ESTUDOS
BY ELEMAR JR

```
public sealed class Singleton
{
    private static Singleton _instance;

    private Singleton() { }

    public static Singleton Instance
    {
        get
        {
            if (_instance is null)
                _instance = new Singleton();
            return _instance;
        }
    }
}
```

A intenção do singleton pattern é garantir que a classe tenha apenas uma instância, provendo acesso global a ela.

Intenção


Cliente 1
Aciona/instancia
Cliente 4
Aciona/instancia
Singleton
Aciona/instancia
Cliente 2
Aciona/instancia
Cliente 3

Abstract Pattern

Singleton Pattern


```
public sealed class Singleton
{
    private static Singleton _instance;
    private static readonly object _lock = new object();

    private Singleton() { }

    public static Singleton Instance
    {
        get
        {
            lock (_lock)
            {
                if (_instance is null)
                    _instance = new Singleton();
                return _instance;
            }
        }
    }
}
```

Singleton Pattern


```
public sealed class Singleton
{
    private static Singleton _instance;
    private static readonly object _lock = new object();

    private Singleton() { }

    public static Singleton Instance
    {
        get
        {
            lock (_lock)
            {
                if (_instance is null)
                    _instance = new Singleton();
                return _instance;
            }
        }
    }
}
```

Classe não pode ser herdada

Singleton Pattern

Campo para controle de concorrência

```
public sealed class Singleton
{
    private static Singleton _instance;
    private static readonly object _lock = new object();

    private Singleton() { }

    public static Singleton Instance
    {
        get
        {
            lock (_lock)
            {
                if (_instance is null)
                    _instance = new Singleton();
                return _instance;
            }
        }
    }
}
```

Singleton Pattern

CLUBE DE ESTUDOS
BY ELEMAR JR

Uma *thread* possui acesso por vez, garantindo que um objeto apenas seja instanciada

Realiza o *lock*, garantindo que a operação será *thread safe*

```
public sealed class Singleton
{
    private static Singleton _instance;
    private static readonly object _lock = new object();

    private Singleton() { }

    public static Singleton Instance
    {
        get
        {
            lock (_lock)
            {
                if (_instance is null)
                    _instance = new Singleton();
                return _instance;
            }
        }
    }
}
```

Possui o ônus de performance

Singleton Pattern


```
public sealed class Singleton
{
    private static Lazy<Singleton> _lazyInstance = new(() => new Singleton());

    private Singleton() { }

    public static Singleton Instance
    {
        get
        {
            return _lazyInstance.Value;
        }
    }
}
```

Carrega a instância como *Lazy*

Singleton Pattern


```
public sealed class Singleton
{
    private static Lazy<Singleton> _lazyInstance = new(() => new Singleton());

    private Singleton() { }

    public static Singleton Instance
    {
        get
        {
            return _lazyInstance.Value;
        }
    }
}
```

É *thread safe* e faz todas as tratativas necessárias

Não atende o Single Responsibility Principle, uma vez que o objeto é responsável de gerenciar a sua vida útil, além de fazer o que se propõe

Se possuir mais de uma Singleton class, precisará duplicar a lógica necessária para a gestão da vida útil do objeto

Em aplicações com injeção de dependência, o uso do AddSingleton permite que o container gerencie o ciclo de vida do objeto, adotando o comportamento de Singleton sem precisar implementar explicitamente o Singleton Pattern.

```
builder.Services.AddSingleton<Singleton>();
```

CAPÍTULO 2

Structural Design Patterns

Conjunto de padrões que oferecem formas de compor objetos, usando herança e outros métodos, visando facilitar a inclusão de novas funcionalidades.

DECORATOR

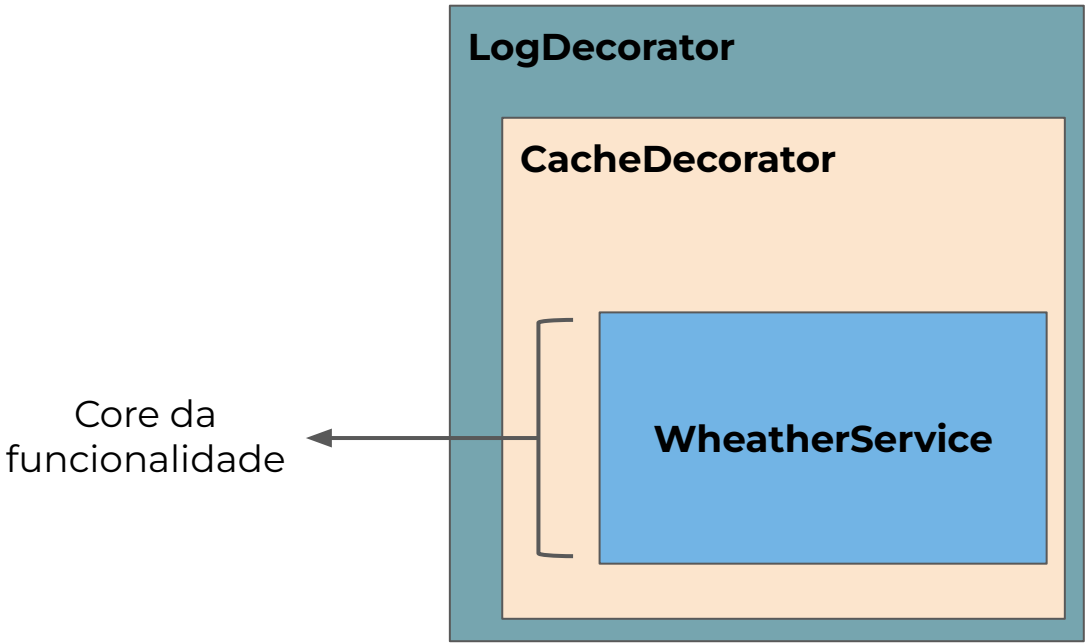
Que dores vamos enfrentar ao realizar manutenção neste código?

CLUBE DE ESTUDOS
BY ELEMAR JR

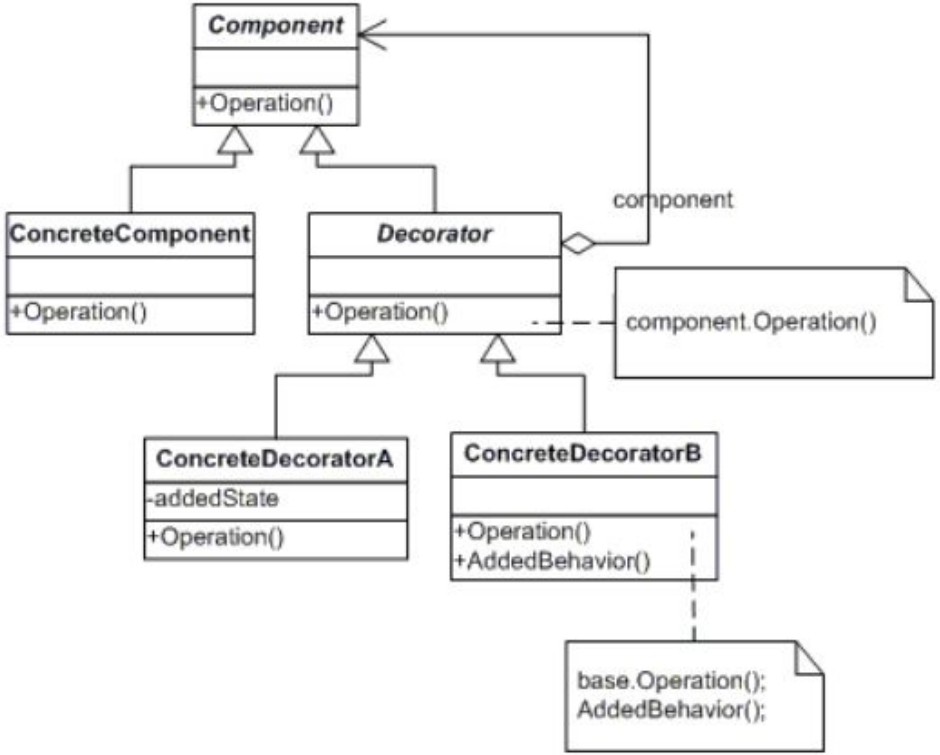
```
public async Task<string> GetWeatherAsync(string city)
{
    try
    {
        var response = await $"http://api.openweathermap.org/data/2.5/weather"
            .SetQueryParams(new { q = city, appid = _apiKey, units = "metric" })
            .AllowAnyHttpStatus()
            .GetAsync();

        if (response.StatusCode == 200)
            return await response.GetStringAsync();

        return $"Error: {response.StatusCode}";
    }
    catch (FlurlHttpException ex)
    {
        return $"Request failed: {ex.Message}";
    }
}
```

Adicionar novos comportamentos a objetos de maneira dinâmica ao colocá-los dentro de objetos especiais chamados decoradores. Este padrão é uma alternativa flexível à herança para estender as funcionalidades de uma classe.

Intenção


LogDecorator
CacheDecorator
WheatherService
Core da
funcionalidade

Decorator


Component
+Operation()
component
ConcreteComponent
+Operation()
Decorator
+Operation()
component.Operation()
ConcreteDecoratorA
-addedState
+Operation()
ConcreteDecoratorB
+Operation()
+AddedBehavior()
base.Operation();
AddedBehavior();

Component


```
public interface IWeatherService
{
    Task<string> GetWeatherAsync(string city);
}
```

Concrete Component


```
public class WeatherService : IWeatherService
{
    public async Task<string> GetWeatherAsync(string city)
    {
        try
        {
            var response = await $"http://api.openweathermap.org/data/2.5/weather"
                .SetQueryParams(new { q = city, appid = _apiKey, units = "metric" })
                .AllowAnyHttpStatus()
                .GetAsync();

            if (response.StatusCode == 200)
                return await response.GetStringAsync();

            return $"Error: {response.StatusCode}";
        }
        catch (FlurlHttpException ex)
        {
            return $"Request failed: {ex.Message}";
        }
    }
}
```

Implementação do componente core que detém a funcionalidade

Concrete Decorator


```
public class WeatherServiceWithCacheDecorator : IWeatherService
{
    private readonly IWeatherService _innerWeatherService;

    public WeatherServiceWithCacheDecorator(IWeatherService innerWeatherService)
    {
        _innerWeatherService = innerWeatherService;
    }

    public async Task<string> GetWeatherAsync(string city)
    {
        // Caching operation
        var result = await _innerWeatherService.GetWeatherAsync(city);
        return result;
    }
}
```

Implementação do decorator

Concrete Decorator


```
public class WeatherServiceWithCacheDecorator : IWeatherService
{
    public WeatherServiceWithCacheDecorator(IWeatherService innerWeatherService)
    {
        _innerWeatherService = innerWeatherService;
    }

    public async Task<string> GetWeatherAsync(string city)
    {
        // Caching operation
        var result = await _innerWeatherService.GetWeatherAsync(city);
        return result;
    }
}
```

Injeção do objeto interno

Concrete Decorator


```
public class WeatherServiceWithCacheDecorator : IWeatherService
{
    public WeatherServiceWithCacheDecorator(IWeatherService innerWeatherService)
    {
        _innerWeatherService = innerWeatherService;
    }

    public async Task<string> GetWeatherAsync(string city)
    {
        // Do caching
        var result = await _innerWeatherService.GetWeatherAsync(city);
        return result;
    }
}
```

Chamar componente interno

Decorator com o uso da biblioteca Structor


```
services.Decorate<IWeatherService, WeatherServiceWithCache>();
```

FACADE PATTERN

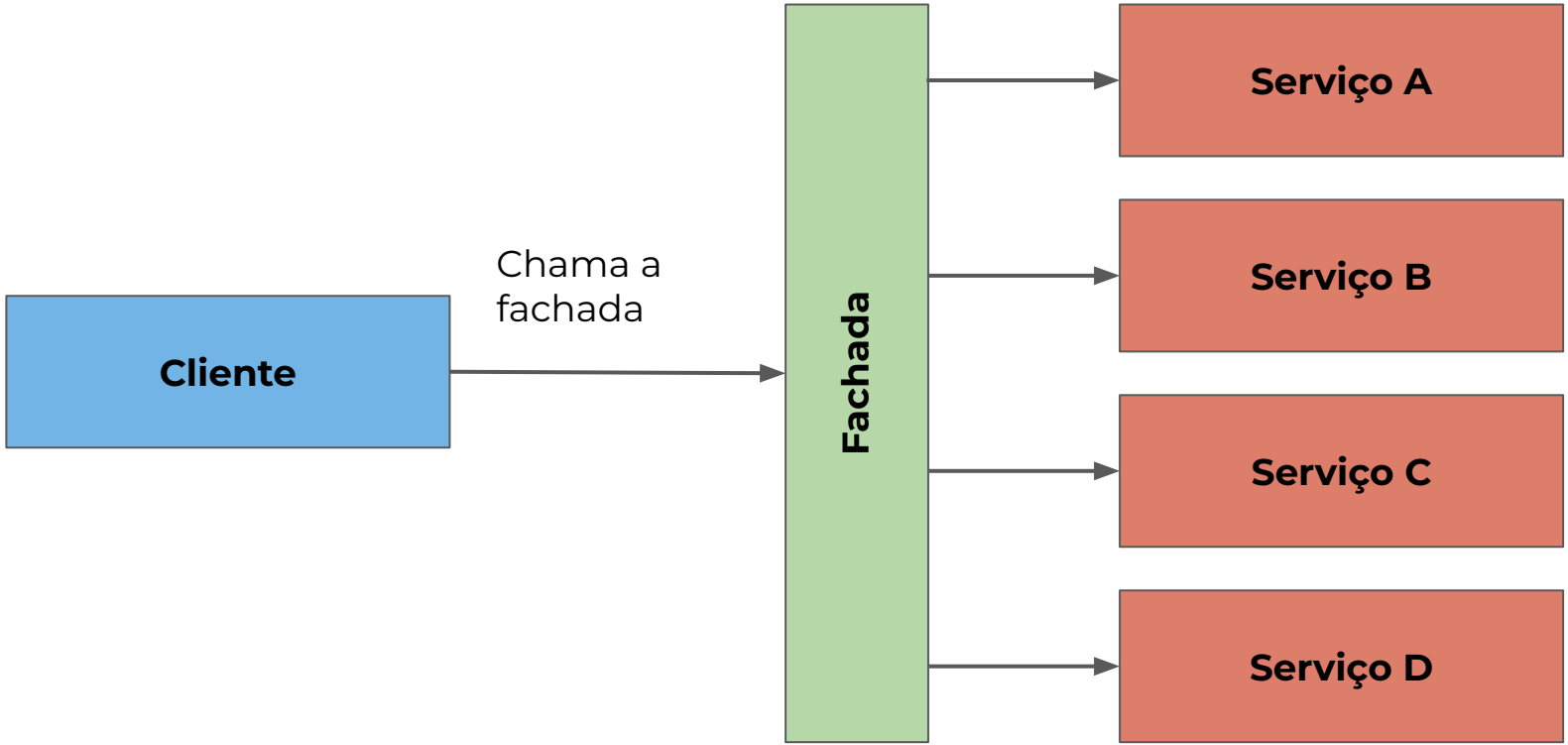
Que dores vamos enfrentar ao realizar manutenção neste código?

CLUBE DE ESTUDOS
BY ELEMAR JR

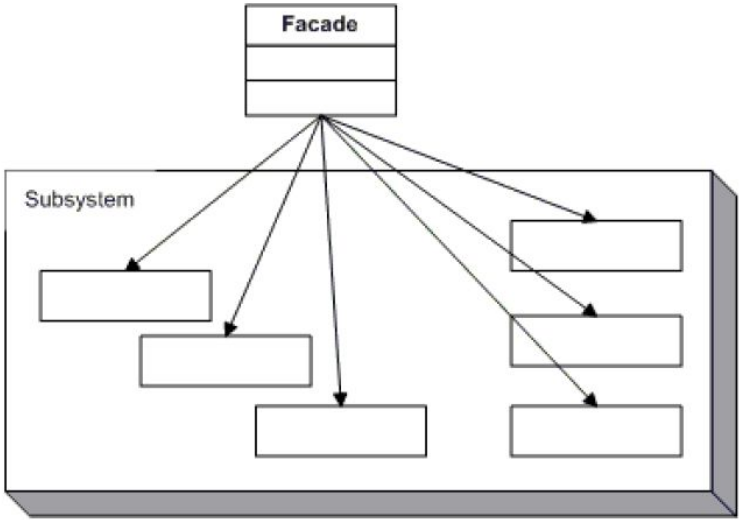
```
Customer customer = new("Francisco Leonardo Schneider");

Bank bank = new();
var isEligible = bank.HasSufficientSavings(customer, 12500);

Credit credit = new();
isEligible = credit.HasGoodCredit(customer);

Loan loan = new();
isEligible = loan.HasNoBadLoans(customer);
```

A intenção do facade pattern é prover uma interface única para um grupo de interfaces em um subsistema. Ele define uma interface de alto nível, visando facilitar o consumo desse subsistema pelos clientes.

Intenção


Cliente
Chama a fachada
Fachada
Serviço A
Serviço B
Serviço C
Serviço D

Abstract Factory



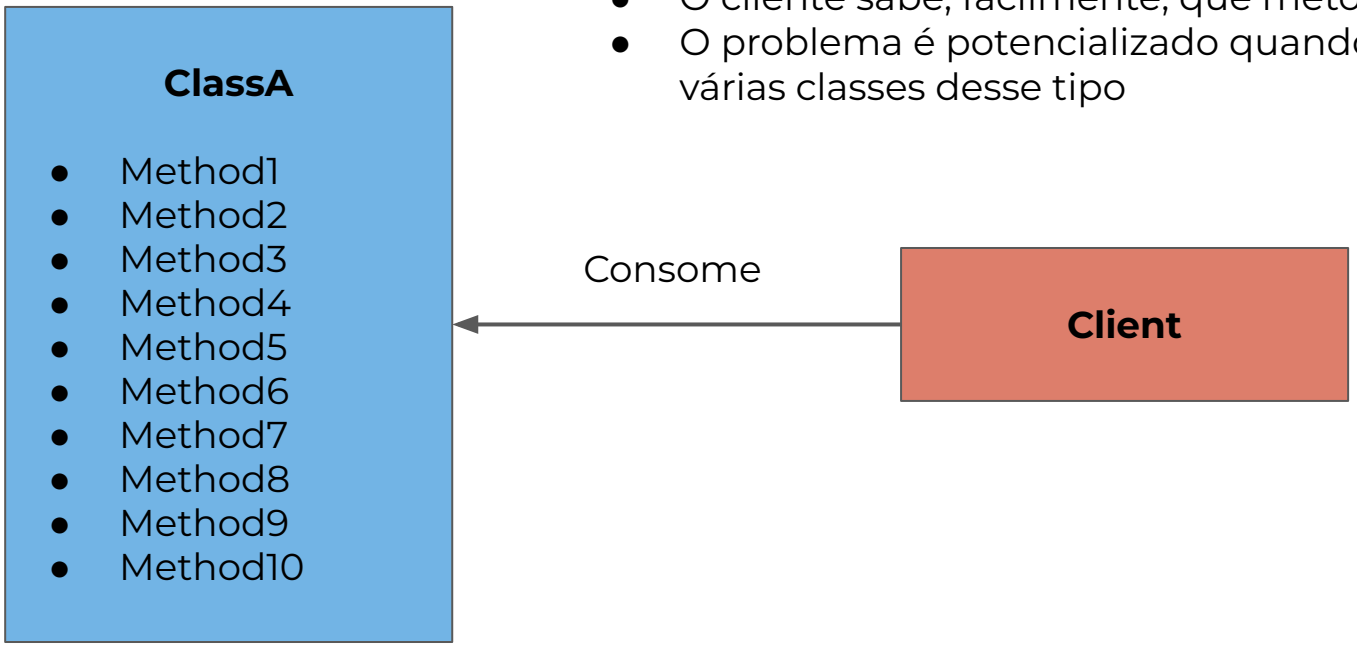
Facade


```
public class FinancingFacade
{
    private readonly Bank _bank;
    private readonly Loan _loan;
    private readonly Credit _credit;

    public FinancingFacade(Bank bank, Loan loan, Credit credit)
    {
        _bank = bank;
        _loan = loan;
        _credit = credit;
    }

    public bool IsEligible(Customer customer, int amount)
    {
        if (!_bank.HasSufficientSavings(customer, amount))
            return false;

        if (!_loan.HasNoBadLoans(customer))
            return false;

        if (!_credit.HasGoodCredit(customer))
            return false;

        return true;
    }
}
```

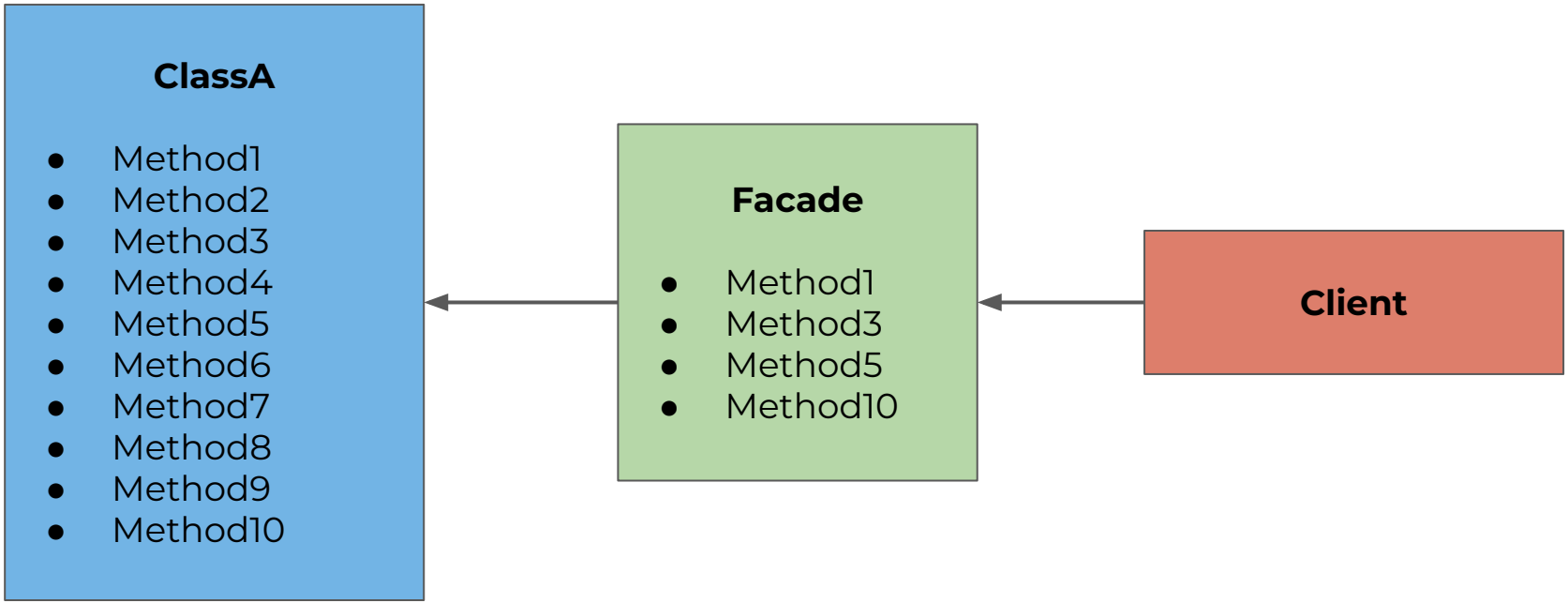
Estrangulando o legado com Facade Pattern

Intenção

- O cliente sabe, facilmente, que método utilizar?
- O problema é potencializado quando existem várias classes desse tipo



Intenção


ClassA
Method1
Method2
Method3
Method4
Method5
Method6
Method7
Method8
Method9
Method10
Facade
Method1
Method3
Method5
Method10
Client

Protege os clientes dos subsistemas, uma vez que o número de objetos que eles precisam lidar é menor

Atende ao open/closed principle, pois o cliente se relaciona apenas com a fachada, reduzindo o acoplamento com os subsistemas

Os clientes continuam podendo acessar diretamente os subsistemas

Facade no EF Core


```
public class DatabaseFacade :
    IInfrastructure<IServiceProvider>,
    IDatabaseFacadeDependenciesAccessor,
    IResettableService
{
    private readonly DbContext _context;
    private IDatabaseFacadeDependencies? _dependencies;

    // Rest of the class
}
```

CLUBE DE ESTUDOS
BY ELEMAR JR